

User Mode Linux Redesign

Architecture, Implementation State, and Validation Evidence

Reporting cutoff: branch `umlctl1-deploy`, commit `9ccbf5e7bb58`

May 19, 2026

Abstract

This report summarizes the User Mode Linux redesign branch as of May 19, 2026. It is a state report for maintainers: the first pages state the branch claim, the delta from inherited UML, and the current validation position; later sections provide design detail, benchmark data, validation evidence, and remaining work. The central claim is that the redesign has moved beyond paper architecture: backend abstraction, static-key gates, profile work, the Rust launcher, and a substantial `kvm-v2` backend are present in-tree. The remaining formal gate is sustained validation and upstream packaging.

1 Executive Summary

1.1 Goal and Current Position

This branch proposes a profile-driven direction for UML while preserving the existing contract: the upstream Linux kernel built as `arch/um` and executed as a host process. The target is not another virtual machine monitor, not a container runtime, and not a Linux reimplementation. The branch-local work is the combination of swappable execution backends, runtime instrumentation gates, profile-driven builds, and validation artifacts from one source tree.

Branch claim. One kernel tree can produce fast production UML, sanitizer-heavy research UML, fuzzing UML, sandbox UML, library-mode UML, and time-travel UML.

Architecture. Layer 1 abstracts the trap mechanism with `struct um_backend_ops`; Layer 2 places static-key gates on hot paths; Layer 3 composes compile-time sanitizers and profile features.

Current state at May 19, 2026. Backend abstraction, static-key infrastructure, profile work, `umlctl`, selftests, and `kvm-v2` are in-tree. `kvm-v2` has working guest entry, syscall dispatch, exceptions, signals, SMP state repair, post-gadget performance, snapshot+record/replay primitives, and declarative host resource controls. The Round-14 Django cache-flake (1–2% under sustained iteration) was root-caused and fixed (SMP-T73, YMM-upper XSAVE leak).

Evidence. `kvm-v2` matches `seccomp` on the substrate gate, reaches cpython-parity 21/21 under UP and SMP, passes the in-tree `umlctl mission` acceptance gate end-to-end (6/6 phases, 100/100 soak, ~15 min), and is faster than `seccomp` on all eight benchmark hosts. Snapshot bench: capture 38.9 μ s, median restore 13 μ s (~1300 $\times$ / ~75 $\times$ under the memo-12 targets).

Figure 1: The report in four claims. Each claim maps back to a tracked source document or measured artefact.

1.2 Existing UML Contract and Branch Questions

The branch assumes the existing UML contract: Linux itself runs as a userspace process, remains debuggable and rootless, and can host isolated Linux instances without emulating a full machine [1, 2, 3]. The proposed changes keep that contract and concentrate on branch-local pressure points: userspace trap cost, multiprocessor state ownership, profile composition, and validation breadth.

1.3 Cutoff Snapshot

1.4 What Is Done, What Is Not

1.5 First Three-Page Takeaway

The branch is best understood as a transition from one default execution story to an explicit profile matrix. The common lower layer is a backend contract; production can choose KVM, CI can choose `seccomp`, legacy hosts can choose `ptrace`, and each profile can decide whether it wants no hooks, runtime hooks, or heavy compile-time instrumentation.

Table 1: Existing UML contract mapped to the branch-local response.

Existing property	Constraint carried forward	Branch-local response
Kernel as a normal host process	Preserve rootless development, CI usability, and reproducible debugging where nested virtualization is unavailable.	Keep stock-host execution through ptrace/seccomp, but add KVM as a fast backend when available.
Isolated Linux instances	Preserve cheap Linux environments without new hardware or privileged host setup.	Make sandbox a profile, not the whole architecture; choose minimal code, seccomp-only defaults, and explicit validation.
Faithful upstream kernel	Preserve Linux behavior rather than introducing a userspace reimplementaion with semantic drift.	Build the upstream kernel as UML; profiles change defaults and instrumentation, not kernel semantics.
Debuggable experimentation	Preserve normal UML debugging value while adding tracing, coverage, and replay hooks as explicit features.	Layer 2 static keys make observability a runtime decision; Layer 3 composes sanitizers by profile.
Known hard problems	Treat trap cost and SMP state ownership as implementation and validation problems, not as presentation claims.	kvm-v2 and sustained validation directly target syscall cost, SMP state ownership, parity, and long-run reliability.



Figure 2: Dashboard metrics at the reporting cutoff. The numbers are intentionally sparse: each is backed by a source table later in the report.

The current branch has crossed the highest-risk implementation path: **kvm-v2** can run real guest userspace, handle SMP, preserve cross-task state, survive a 14-round real-world flake investigation (Round 14’s YMM-upper-128 XSAVE leak under AVX un-mask, fixed as SMP-T73), beat **seccomp** on measured workloads, snapshot and replay its own state, and accept declarative host resource controls.

A single command — `umlctl mission --kernel <path>` — now produces a binary acceptance verdict in ~15 min wall: all six phases must pass (KUnit, performance bench, substrate parity, host resource controls, diverse workload soak, diagnostic snapshot). The remaining gate is the wall-clock 24-hour soak and the LKML upstream submission of the **kvm-backend** series.

Table 2: Questions the report uses to bound the branch claims.

#	Question	Where answered
1	What branch-local value proposition is being proposed?	Branch claim, non-goals, existing-contract table, and profile matrix.
2	What is the boundary relative to QEMU, Firecracker, gVisor, LKL, and containers?	Non-goals, design boundary, and adjacent-work comparisons.
3	What single architecture lets fast production, research, fuzzing, sandbox, library, and time-travel coexist?	Three-layer architecture and profile sections.
4	What has actually landed, and what is still only planned?	Current-state tables, source map, and upstream queue.
5	Does <code>kvm-v2</code> preserve Linux-visible state across SMP, signals, FPU/XSAVE, faults, and task switches?	KVM design, state-audit method, recent-fix table, and validation ladder.
6	Does the performance win survive beyond a microbenchmark?	Micro, Python, and stress-tier benchmark sections.
7	What validation bar remains before upstreaming?	Sustained validation, Tier 2/3/LTP, and next-work sections.
8	What should maintainer review put pressure on next?	Open-state list, remaining risks, and upstream-path sections.

Contents

1	Executive Summary	2
1.1	Goal and Current Position	2
1.2	Existing UML Contract and Branch Questions	2
1.3	Cutoff Snapshot	2
1.4	What Is Done, What Is Not	2
1.5	First Three-Page Takeaway	2
2	Goal and Vision	7
2.1	Problem Statement	7
2.2	Non-Goals	7
2.3	Success Criteria	7
2.4	Why Now	7
3	Current State	8
3.1	Source Basis	8
3.2	Inherited UML Versus This Branch	8
3.3	Landed Versus Open	8
3.4	Current Functional Reading	9
3.5	Open State	9
4	Architecture	11
4.1	Three Layers	11
4.2	Layer 1: Backend Contract	11
4.3	Layer 2: Static-Key Hot Paths	11
4.4	Layer 3: Compile-Time Instrumentation	12
4.5	A Syscall Through the Stack	12
5	KVM v2 Design	12
5.1	Model	12

Table 3: Reporting snapshot.

Reporting date	May 19, 2026
Cutoff commit	9ccbf5e7bb58 on umlctl-deploy
Project corpus	255+ files under Documentation/virt/uml/redesign/; 32 top-level tools/testing/selftests/um/ test areas; arch/um/backend/kvm-v2/ implementation present including snapshot, record, state-trace, lstar-gadget, byteshape and marshal KUnit
Functional headline	kvm-v2 has working guest entry, syscall dispatch, exceptions, signals, SMP state repair, post-gadget performance, snapshot + record/replay primitives (KUnit 3+7 = 10/10), declarative host resource controls (SMP-T78-T84), and a first-class umlctl mission acceptance gate (6/6 phases PASS in ~15 min)
Parity headline	cpython-parity is 21/21 under both UP and SMP; substrate gate is PASS=25/FAIL=3/XFAIL=3, matching seccoMP; cache-flake bug from Round 14 closed via SMP-T73 (5-week investigation, YMM-upper XSAVE save/restore leak under AVX un-mask)
Performance headline	post-gadget kvm-v2 beats seccoMP on all eight lab hosts: micro 193–908x, Python 2.99–4.06x, stress 3.71–8.72x; snapshot capture 38.9 μ s and median restore 13 μ s vs memo-12 targets of <50 ms / <1 ms
Validation headline	umlctl mission (6-phase comprehensive gate): 10/10 KUnit cases, bench gates met, substrate PASS, host_resources end-to-end verified, 100/100 PASS across 5 diverse workloads in ~8 min soak, 0 panics / 0 SIGBUS / 0 high-cr2; 24-hour long-tail soak is the remaining wall-clock evidence

5.2	Lessons from the v1 KVM Backend	12
5.3	Implementation Themes	13
5.4	Design Detail: Main Control Surfaces	14
5.5	Design Detail: Fast Path Versus Slow Path	14
5.6	State-Audit Method	14
5.7	Recent Fixes	15
5.8	The Round-14 Cache-Flake Saga (SMP-T73)	15
5.9	Design Boundary	16
6	Profiles and Validation	16
6.1	Profile Matrix	16
6.2	Validation Ladder	17
6.3	umlctl mission Acceptance Gate (2026-05-19)	17
6.4	Earlier Sustained Validation Pilot (2026-05-14)	17
7	Performance and Benchmarks	17
7.1	Benchmark Method	17
7.2	Native Baseline Context	18
7.3	Micro Tier	18
7.4	Application and Stress Tiers	18
7.5	Detailed Cross-Host Table	19
7.6	Snapshot Benchmark	19
7.7	Performance Interpretation	19

Table 4: Status by major workstream.

ID	Workstream	State	Summary
A	Backend abstraction	DONE	<code>um_backend_ops</code> contract, ptrace/seccomp split, KUnit contract, and documentation landed.
B	Static-key hot paths	DONE	Hot-path gates, debugfs controls, and runtime flip demonstration landed.
C	Profiles and gaps	MOSTLY	Defconfigs and major sanitizer/tracing ports are largely landed; snapshot v2 + record/replay v2 ports now in tree; syzkaller binding remains external.
D	KVM backend	DONE	<code>kvm-v2</code> functional + performant + R14 cache-flake fixed (SMP-T73); snapshot Phase 1–6 and record/replay Phase 1–7 ports landed; host resource controls (SMP-T78–T84) shipped; <code>umlctl mission</code> acceptance gate operational. 24h long-tail soak and Series 7 upstream submission remain.

8	Remaining Work and Upstream Path	19
8.1	Immediate Work	19
8.2	Snapshot, Replay, and Time-Travel Status	19
8.3	Residual Risks	19
8.4	Upstream Queue	19
8.5	Main Message for Maintainers	20
A	Source Map	20
B	Reporting Data Files	22

2 Goal and Vision

2.1 Problem Statement

The branch starts from the existing UML contract: Linux itself is compiled to run as a userspace program, with real kernel semantics and a deployable target in environments where full hardware virtualization is not available. The branch-local problem is to add a reviewable structure for performance, feature composition, and operational validation without changing what UML means.

The proposed bar for this branch is:

- production UML should approach native syscall cost when the host exposes KVM;
- research UML should ship with sanitizers, tracing, kprobes, ftrace, BPF, KGDB, and debug surfaces composed from one tree;
- fuzzing UML should provide KCOV, snapshot/forkserver startup, and a path toward a syzkaller vm/uml backend;
- sandbox UML should compile out debug surfaces and minimize host attack surface;
- time-travel and record/replay should become a profile, not a niche configuration footnote.

2.2 Non-Goals

The design boundary is explicit. It does not reimplement Linux in another language. gVisor is useful prior art for backend structure and KVM execution, but UML keeps Linux semantics by building Linux itself [4]. It is not a microVM competitor to Firecracker [5]; the problem here is kernel research and fuzzing fidelity, not cloud cold-start packaging. It is not a unikernel-only library effort either, although the library profile borrows from LKL where that model is useful [6].

2.3 Success Criteria

The branch succeeds when a developer can select a profile, build it from the same tree, and get an artifact whose performance and instrumentation properties match its intended use:

Table 5: Target profile outcomes.

Profile	Intended outcome
prod-fast	KVM backend, no hooks, near-native syscall path.
prod-with-hooks	Fast production binary with hooks compiled but off; tracing can be enabled after a crash without rebuilding.
research	Debug-friendly UML with tracing, kprobes, sanitizers, and interactive control surfaces.
fuzz / fuzz-deep	KCOV, snapshots, sanitizer coverage, replay-oriented hooks, and syzkaller integration.
sandbox	Minimal debug surface and minimized host-side trust boundary.
library	LKL-style direct-call mode for harnesses that can sacrifice ring-split fidelity.
time-travel	Deterministic time and replay hooks as a first-class shipped configuration.

2.4 Why Now

The timing is tied to concrete prerequisites: seccomp mode exists, SMP support exists, and kernel instrumentation is mature enough to compose around static keys and profile-specific Kconfig. Adjacent projects also supply useful models: gVisor for backend/platform design, crosvm for host process

decomposition [7], syzkaller for fuzzing expectations [8], and upstream kernel policy for AI-assisted development disclosure [9].

3 Current State

3.1 Source Basis

The report is grounded in the branch’s source material: goal and plan documents, landed code, the bug diary, current status, and measured evidence. The main source classes are:

Table 6: How the project corpus maps into this report.

Source class	Representative paths	Use in the report
Plan and vision	00-vision.md, README.md, report-presentation/PLAN.md	Defines the goal, non-goals, architecture narrative, and publishing shape.
Architecture	01-architecture/, 02-workstreams/	Defines layers, workstreams, dependencies, profiles, and sequencing.
Code	arch/um/include/shared/backend.h, arch/um/backend/kvm-v2/, tools/uml/uml-launcher/	Confirms that the report is describing landed implementation, not only design.
Diary and state	STATUS.md, state-audit/, sustained-validation memos	Provides current truth, bug chronology, closure criteria, and remaining work.
Evidence	tools/testing/selftests/um/, benchmark and soak memos	Supplies measured performance, parity, soak, and gate data.

3.2 Inherited UML Versus This Branch

The attribution boundary matters. I checked it by diffing `origin/master...HEAD` across `arch/um`, `tools/uml`, `tools/testing/selftests/um`, and `Documentation/virt/uml`. That diff spans 688 UML-related files, with 154,501 insertions and 823 deletions. The number is a scope check, not an authorship claim: the existing UML model, the Linux kernel, and generic KVM facilities remain inherited foundations.

3.3 Landed Versus Open

The branch contains substantial code and test infrastructure:

- `arch/um/include/shared/backend.h` defines the backend contract used by host-side and kernel-side translation units.
- `arch/um/backend/seccomp/` and `ptrace` integration provide the non-KVM backends.
- `arch/um/backend/kvm-v2/` contains the active `kvm-v2` implementation: vCPU, memory, syscall, exception, region, state-trace, LSTAR gadget; snapshot (Phase 1–6, with `cmdline` + `debugfs` bench harness) and record/replay (Phase 1–7, KUnit 7/7) primitives; KUnit suites for byte-shape, marshal, snapshot, and record.
- `arch/um/os-Linux/` carries the SMP-T78–T82 host-side resource-control levers: `MAP_POPULATE` and `MAP_LOCKED` on `physmem`, `mlockall`, `madvise(MADV_UNMERGEABLE)` on `physmem`, `MADV_HUGEPAGE/NOHUGEPAGE` control, `/proc/self/oom_score_adj` write, optional `MAP_HUGETLB` 2M/1G backing, and process-level `sched_setaffinity`.
- `tools/uml/uml-launcher/` and `umlctl` provide host-side launch, supervision, gate, gate-loop, history, and deployment machinery, plus the new `[host_resources]` TOML section (`cgroup v2`

Table 7: Attribution boundary for the current branch.

Category	What the report means
Inherited UML and Linux	Linux-as-a-host-process, the upstream UML port, legacy ptrace execution, existing kernel subsystems, the host KVM API, and the general idea of deterministic time-travel are not claimed as new work here.
Recent upstream prerequisites	Seccomp-mode UML and upstream SMP work are treated as enabling context. The redesign builds on them; it does not present them as branch-local inventions.
Redesigned in this project	The explicit backend-ops contract, static-key hook model, profile matrix, validation ladder, staged upstream queue, and kvm-v2 state-ownership method are the project architecture.
Implemented in this branch	arch/um/backend/kvm-v2/ including snapshot.c (Phase 1–6 with cmdline+debugfs bench harness), record.c (Phase 1–7 with gadget-bypass , RDTSC , SIGALRM , KUnit 7/7), state-trace , lstar-gadget , byteshape and marshal KUnit suites; backend contract code; profile configs and docs; snapshot/forkserver v1 (C-09); the Rust launcher and umlctl including the new mission subcommand (6-phase acceptance gate), [host_resources] TOML schema, cgroup v2 + preflight modules; KVM/soak/benchmark selftests with the diverse-workload soak daemon; the report/deck workspace.
Planned or paused	24-hour long-tail soak (the operational gate remaining after the umlctl mission 15-min acceptance gate), LKML upstream Series 7 (kvm-backend), syzkaller vm/uml (external), LTP curation , and Tier 3 host resource controls (SMP-T85 SCHED_FIFO , T86 NUMA , T87 O_DIRECT — design memo 52 §3 preserves the design until a concrete operator question demands implementation).

limits + env var translation), **cgroup** module, **preflight** verification module, and the **mission** subcommand (6-phase acceptance gate).

- **tools/testing/selftests/um/** contains **benchmark**, **parity**, **gate**, **soak** (daemon-driven, 11 workload templates each carrying a default **[host_resources]** block), **snapshot**, **KVM record**, **ftrace**, **kprobes**, **KMSAN**, and **process reproducer** coverage.

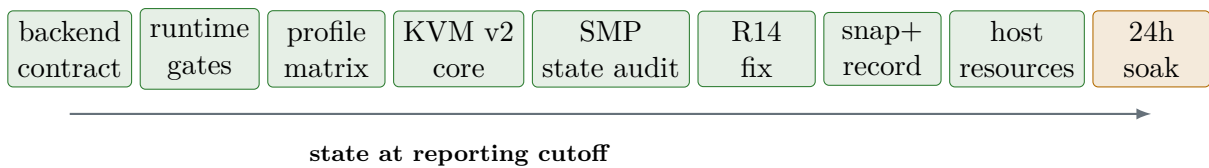


Figure 3: Milestone state. The R14 cache-flake fix (SMP-T73 YMM-upper XSAVE leak), snapshot Phase 1–6, record/replay Phase 1–7, and host-resource controls (SMP-T78–T84) all landed since the previous reporting cutoff. The remaining open lane is the 24-hour wall-clock soak.

3.4 Current Functional Reading

3.5 Open State

The important open items are bounded:

- The 15-minute **umlctl mission** acceptance gate passes 6/6 phases including a 100/100 diverse-workload soak. The 24-hour continuous run remains as the long-tail wall-clock evidence; Tier 2/Tier 3 templates and LTP curation are wired into the soak daemon and need long-running confirmation.

Table 8: Functional status from `STATUS.md`.

Surface	Reading	Interpretation
<code>init=/bin/echo</code>	UP + SMP yes	Basic guest entry works under <code>kvm-v2</code> .
Python smoke	UP + SMP yes	<code>python3 -V</code> , <code>hashlib</code> , <code>ssl</code> , and single imports work.
Substrate gate	PASS=25, FAIL=3, XFAIL=3	<code>kvm-v2</code> matches <code>seccomp</code> on the curated substrate gate.
<code>cpython-parity</code>	21/21	The old P0, Python C-extension import crashes, is functionally closed.
<code>mt-mini T=8, ncpus=4</code>	30/30 or better in current runs	SMP memory stress reaches the current acceptance bar.
<code>threaded-fork-malloc</code>	0/24000 child failures	Cross-task XMM/FPU guarantee survived the T55/T57 fixes.
<code>wide cpython-parity</code>	134 parity, 0 regression	Larger curated module sweep found no <code>kvm-v2</code> regression.
Round-14 Django cache-flake	closed (SMP-T73)	1–2% bytecode-page YMM-upper-128 corruption fixed by switching 5 per-task FPU save/restore sites from <code>KVM_GET_FPU</code> to <code>KVM_GET_XSAVE</code> . 220/220 PASS validation, $p=0.007$ vs baseline.
Snapshot/RR KUnit	10/10 cases	3 snapshot (basic, full, task) + 7 record/replay (basic, state, observe, strict_replay, gadget_bypass, rdtsc, sigalrm).
Snapshot bench	38.9 μ s capture, 13 μ s median restore	Memo-12 targets <50 ms / <1 ms cleared by $\sim 1300\times$ and $\sim 75\times$ respectively.
<code>umlctl mission acceptance gate</code>	6/6 phases PASS, ~ 15 min wall	KUnit + bench + substrate + <code>host_resources</code> + 100/100 soak + diagnostic, 0 panics / 0 SIGBUS / 0 high-cr2 anomalies.

- Snapshot Phase 1–6 and record/replay Phase 1–7 are in tree (memo 26 + memo 27); the remaining record/replay work is the host-side signal-interception wiring that turns a real SIGALRM into an `observe_sigalrm` call (the log shape and replay-side gating are already proven by KUnit).
- The R14 Round-14 cache-flake closure (SMP-T73 YMM-upper XSAVE leak, plus T74/T75/T76 latent-bug follow-ups and the T77 feature-enablement checklist) is landed and validated. AVX-512 / AMX / PKRU / CET / AVX-VNNI un-mask remains future work; memo 52 §1–§2 documents the prerequisite check-list and the host-side levers shipped to ease it.
- The LKML queue is sequenced; Series 7 (kvm-backend) is now backend-layer unblocked by R14 closure and the new acceptance gate, pending cover-letter refresh and submission.
- Tier 3 host-resource features (`SCHED_FIFO` RT, NUMA binding, `O_DIRECT` for hostfs I/O) are deferred per memo 52 §3: the design is preserved, implementation opens only when a concrete operator question demands it (multi-tenant CI host, multi-socket bench rig, multi-UML memory-pressure soak).

4 Architecture

4.1 Three Layers

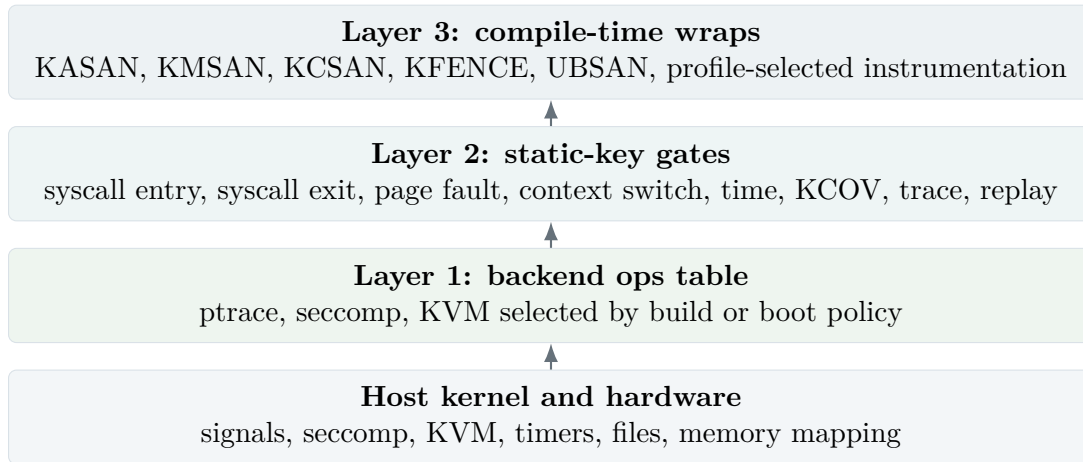


Figure 4: The architectural decomposition. Lower layers do not know about higher-layer policy; profiles choose points in the product space.

Layer 1 isolates the mechanism that makes guest userspace run: `ptrace`, `seccomp`, or KVM. Layer 2 isolates optional runtime observation and policy hooks. Layer 3 isolates compiler-level instrumentation whose cost is inherent in the binary. This structure matches the host kernel’s own composition strategy: explicit lower interfaces, static keys for runtime feature toggles, and Kconfig for compile-time feature selection.

4.2 Layer 1: Backend Contract

The live contract is `struct um_backend_ops`. It has metadata, capability flags, lifecycle hooks, vCPU execution, memory region callbacks, scheduling hooks, TLB kick support, time hooks, and debug/introspection hooks. The important design choices are:

- every in-tree backend populates the whole table; unsupported cold operations return an explicit error rather than leaving NULL fields;
- single-backend builds can compile calls down to direct calls; dynamic builds can select at boot and pay one stable indirect call;
- backend capability flags replaced legacy global booleans, so host-side code asks the selected backend what stub, reaper, and syscall behavior it needs;
- per-mm memory state is owned by the backend, which lets `seccomp` keep its stub model while `kvm-v2` manages memslots and guest mappings.

4.3 Layer 2: Static-Key Hot Paths

Layer 2 is the answer to a central tension: a research build wants hooks everywhere; a production build wants the hot path cold and small. Static keys make the disabled case a patched NOP sequence and the enabled case an explicit branch to slow-path code. The off-state cost model is therefore stable and auditable.

The report treats Layer 2 as a policy plane rather than a pile of tracepoints. The same hot path can support tracing, KCOV, record/replay, time-travel, and performance event emission, provided each hook has a clear default state and each profile declares which hooks are compiled and enabled.

Table 9: Backend contract categories.

Category	Representative ops	Why it belongs in Layer 1
Lifecycle/trap	<code>probe</code> , <code>init</code> , <code>vcpu_run</code>	Backend availability and guest execution are mechanism-specific.
Memory	<code>mm_create</code> , region add, remove, protect	<code>ptrace/seccomp</code> stubs and KVM memslots have different state models.
Scheduling	thread create, context switch, IPI	Guest scheduling touches backend-owned vCPU or stub state.
Time	clock read, timer set	Determinism and backend timing sources must be abstracted.
Debug	register read/write	KGDB and inspection need a common contract over different mechanisms.

4.4 Layer 3: Compile-Time Instrumentation

Layer 3 includes KASAN, KMSAN, KCSAN, KFENCE, UBSAN, and related compiler- or allocator-level instrumentation. These features cannot be made free by a runtime branch. They change code generation, memory layout, or allocator behavior. The architecture therefore treats them as profile-level build choices that compose with any backend below.

4.5 A Syscall Through the Stack

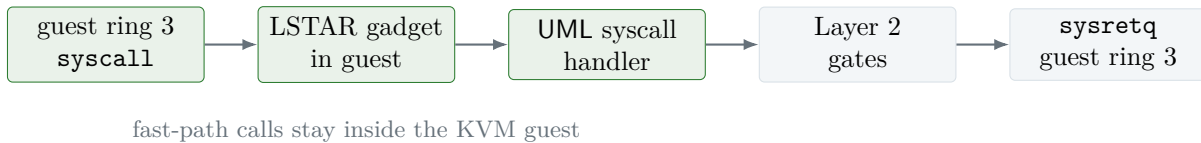


Figure 5: `kvm-v2` fast syscall shape after gadget revival. Trivial syscalls avoid a host `vmexit` on the hot path.

5 KVM v2 Design

5.1 Model

`kvm-v2` follows the gVisor KVM-platform idea, adapted to UML: guest userspace runs at ring 3 inside a KVM guest, the UML kernel runs at ring 0 inside the same KVM guest, and ordinary syscalls enter the UML kernel through `MSR_LSTAR` instead of trapping to a host monitor [4, 10]. Host exits are still used for operations that truly require host participation, but the target is to keep the common syscall path in guest context.

5.2 Lessons from the v1 KVM Backend

The `kvm-v2` design is not just a cleaned-up version of the first KVM implementation. The v1 implementation was valuable because it proved the fast trap path was possible, but it also showed that the initial data model would not survive sustained SMP execution. The diary around the April 27 architecture review and the v1 restart guide converged on the same lesson: treating a KVM vCPU as a passive trap resource shared by UML scheduler machinery creates state ownership bugs that cannot be made impossible by adding more repair code.

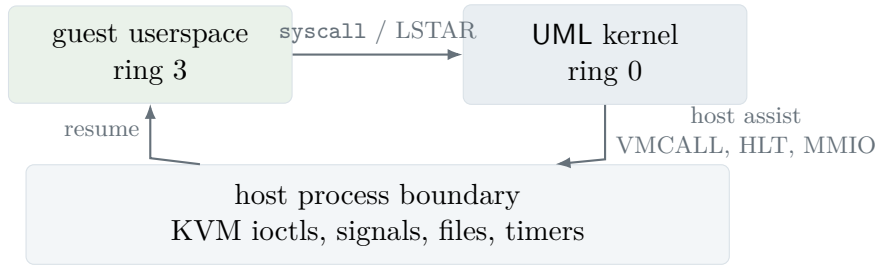


Figure 6: `kvm-v2` execution model. It is not a QEMU-style machine; it is a trap mechanism for UML itself.

Table 10: Design lessons carried from KVM v1 into `kvm-v2`.

v1 pressure	Failure mode learned	<code>kvm-v2</code> response
KVM as a faster trap loop	The per-vCPU <code>kvm_run</code> mmap and architectural state have active ownership rules; they are not just a scratch buffer behind a syscall trap.	Make ownership explicit: per-vCPU state, per-task saved state, per-mm state, and synchronization rules are documented and audited.
Repair on next entry	Snapshotting exit state and repairing later does not compose with <code>-EINTR</code> , host signals, and SMP scheduling.	Use explicit entry/exit boundaries, KVM-visible signal masking, and state trace instead of relying on opportunistic repair.
Shadow page-table machinery	Multiple producers had to keep a shadow view coherent while KVM consumed it; every new fix added another ordering rule.	Keep the v2 memory contract smaller and make TLB generation, memslot policy, and per-vCPU refresh rules visible.
Implicit scheduler assumptions	<code>preempt_disable()</code> did not mean “this task cannot move to another host CPU” for the KVM state being touched.	Use <code>migrate_disable()</code> at the dispatch and capture sites that depend on a selected per-host-CPU vCPU.
Ad hoc bug hunting	Individual hypotheses produced regressions or negative results because the state space was not enumerated.	The state-audit method enumerates state items, operations, owners, save/restore rules, and suspect registers before accepting a fix.

This is the changed contract for maintainer review: v2’s core claim is not only that KVM can run UML faster, but that each architectural state item has a declared home and a declared transition rule.

5.3 Implementation Themes

The current `kvm-v2` design is organized around five hard problems:

1. **Guest entry and return.** The backend must enter guest ring 3, handle `SYSCALL/SYSRET` transitions, and recover when host signals interrupt critical regions.
2. **Per-vCPU and per-task state.** General registers, CR2, segment bases, FPU/XSAVE, pending events, IST frames, and KVM run state cannot leak across tasks or host CPUs.
3. **Memory and TLB state.** UML page-table changes must be reflected into KVM-visible mappings and cross-vCPU TLB state without stale translations.
4. **SMP correctness.** Host scheduling, guest scheduling, signal delivery, and KVM vCPU ownership must compose without assuming that `preempt_disable()` pins a host thread.

5. **Performance.** The backend must preserve the win from in-guest dispatch while avoiding correctness regressions from lazy state handling.

5.4 Design Detail: Main Control Surfaces

Table 11: `kvm-v2` implementation surfaces to understand before reviewing patches.

Surface	Representative code	Design role
Initialization and capabilities	<code>init.c</code> , <code>ops.c</code> , <code>Kconfig</code>	Probe host KVM support, wire backend ops, expose configuration choices.
vCPU execution	<code>vcpu.c</code>	Owns <code>KVM_RUN</code> , register marshaling, migration pinning, FPU capture, dispatch, and per-vCPU state.
Syscall path	<code>syscall_trap.c</code> , <code>lstar_gadget.S</code>	Handles <code>LSTAR</code> entry, in-guest fast syscalls, slow-path exits, and return-state repair.
Memory and regions	<code>memslot.c</code> , <code>region.c</code> , backend memory ops	Maps UML memory changes into KVM-visible regions and memslot policy.
Exceptions	<code>exception.c</code>	Installs and handles <code>IDT/TSS/IST</code> paths, page faults, and special recovery paths.
State tracing	<code>state_trace.[ch]</code>	Gives low-overhead, static-key-controlled visibility into state ownership bugs.
Tests	<code>test_marshall.c</code> , <code>test_byteshape.c</code>	Protect register ABI, gadget byte shape, and internal assumptions.

5.5 Design Detail: Fast Path Versus Slow Path

The backend now has two different syscall cost classes. Trivial syscalls covered by the `LSTAR` gadget stay inside the KVM guest and return directly to guest userspace. Syscalls outside the gadget set, or paths requiring host assistance, still exit to the host and run the full backend machinery. The project should keep these paths described separately:

- **Fast path:** `getpid`-family and clock-like calls that can be answered from guest-visible state with preserved ABI registers and no host-side work.
- **Slow path:** `mmap`, `munmap`, `brk`, I/O, scheduling, signal, and fault paths that need backend state or host services.
- **Recovery path:** interrupted `LSTAR`, page-fault stub, `#NM`, and signal delivery edges where the backend must rebuild user-visible architectural state exactly.

5.6 State-Audit Method

The state-audit directory is part of the design, not just project diary. It converted a vague SMP corruption class into a finite audit: 149 state items, 50+ operations, an ownership matrix, suspect register audits, runtime tracing, and ranked findings.

The most important fix from this arc was the SMP-T13 `migrate_disable()` finding. UML was using `preempt_disable()` to pin access to per-host-CPU vCPU state, but the build did not have preemption count semantics, so the call did not actually prevent migration. Replacing that assumption with `migrate_disable()` removed a structural cross-vCPU ownership violation.

Table 12: State-audit layers.

Layer	Artefact	Contribution
1	State inventory	Enumerates register, control, segment, MSR, FPU, event, descriptor, per-task, per-vCPU, and per-mm state.
2	Operations inventory	Lists hardware, KVM, backend, scheduler, and signal operations that read or write state.
3	Ownership matrix	States who owns each state item at each transition.
4	Suspect register audits	Tracks RBP, RAX, RDX, FS_BASE and similar corruption signatures.
5	Toolkit	cscope, ripgrep, ast-grep, bpftrace, ftrace, and parser recipes.
6–8	Findings and fixes	Converts candidates into patches and regression matrices.

5.7 Recent Fixes

5.8 The Round-14 Cache-Flake Saga (SMP-T73)

This subsection is the closest thing the project has to a teaching case study: a 1–2% corruption rate, fourteen rounds of hypothesis-elimination, an unrelated bug discovered and shipped along the way, an honest false-positive root-cause claim that had to be retracted, and a final fix that landed in a single `ioctl` substitution.

Symptom. A subset of soak iterations of the no-network Django stdlib loopback workload exhibited a 1–2% rate of `Kernel panic` signatures rooted in CPython bytecode pages. The corruption shape was specific: 16-, 24-, 26-, or 40-byte zero regions interspersed with valid bytecode. `kvm-v2` only; `seccomp` clean.

Investigation arc. Round 1–12 worked through and eliminated: KSM page merging, khugepaged collapse, kswapd reclaim, host pagecache demotion, fast-page-fault races, TLB-coherence gaps, `mmu_notifier` traffic, UML self-unmap, physmem residency, CPython interpreter specialization, and the LSTAR-gadget bounds-check path (Round 13 shipped SMP-T68 as an independent fix surfaced by the investigation). Round 13 also disproved a working hypothesis about `MAP_POPULATE` + `MAP_LOCKED` pinning of physmem (SMP-T71) as the bug cause — the pinning still landed as foundation infrastructure but did not affect the rate.

Root cause (Round 14 H17/H18). The corruption shape matched a half-AVX2 vector: 16 valid low bytes followed by 16 leftover-zero upper bytes. SMP-T57 Phase A had un-masked AVX in `CPUID` and set `XCR0 = FP|SSE|YMM = 0x7`. glibc’s `IFUNC` dispatch then started selecting AVX2 `vmovdqu ymm memcopy` variants. But `kvm-v2`’s per-task FPU save/restore (T26/T27 era) used `KVM_GET_FPU` / `KVM_SET_FPU` — the legacy 512-byte FXSAVE area, which covers x87 and XMM low 128 but *not* YMM upper 128. Across a per-host-CPU vCPU pool the upper halves of YMM registers leaked between tasks; an inopportune `memcpy` then wrote 16 user bytes followed by 16 leftover-zero upper-half bytes.

Fix. SMP-T73 substituted `KVM_GET_XSAVE` / `KVM_SET_XSAVE` (4 KB `struct kvm_xsave`, covers YMM upper bytes) at all five per-task save/restore sites in `arch/um/backend/kvm-v2/vcpu.c`. The matching field in `arch_thread.kvm_v2` was retyped from `struct kvm_fpu` to `struct kvm_xsave`.

Validation. 220/220 soak PASS at first repro ($p = 0.007$ vs baseline). 100/100 across five diverse workloads in the subsequent `umlctl mission` acceptance gate. A premature “identified” claim made

mid-investigation (commit 714969f625f6) had to be retracted: the contaminated H17 soak ran the workload-template with mid-run overrides reverted by a parallel git checkout. The honest correction shipped in subsequent commits.

Lessons preserved as infrastructure. Three follow-on artefacts were extracted from the saga:

- SMP-T74/T75/T76 closed three latent bugs with the same architectural shape (per-host-CPU vCPU pool, cross-task state hand-off): DEBUGREGS not pinned, VCPU_EVENTS not saved per task, CPUID leaf 0xD sub-leaves inconsistent with the un-masked feature bits.
- SMP-T77 (memo state-audit Layer 26) is a forward-looking feature-enablement checklist: every future un-mask in `kvm_v2_curate_cpuid` comes with a checklist of what other code must change. The next un-mask cannot re-walk the same gap.
- Memo 52 (host resource controls) is the broader response: if any future regression has both a guest-side bug and a host-induced trigger, the host-side levers (pinning, KSM-disable, THP control, hugepage backing, CPU affinity, cgroup limits, preflight) reduce the noise the investigation has to fight. Memo 52’s lessons-learned section is explicit that these levers did not cause or fix R14 — they make the *next* R14 cheaper.

5.9 Design Boundary

The current backend should be presented upstream as a backend for UML, not as a general VMM. The VMM-like machinery exists only to make UML execution fast and faithful. That framing matters: it keeps device emulation, boot firmware, hardware discovery, and cloud microVM expectations out of scope.

6 Profiles and Validation

6.1 Profile Matrix

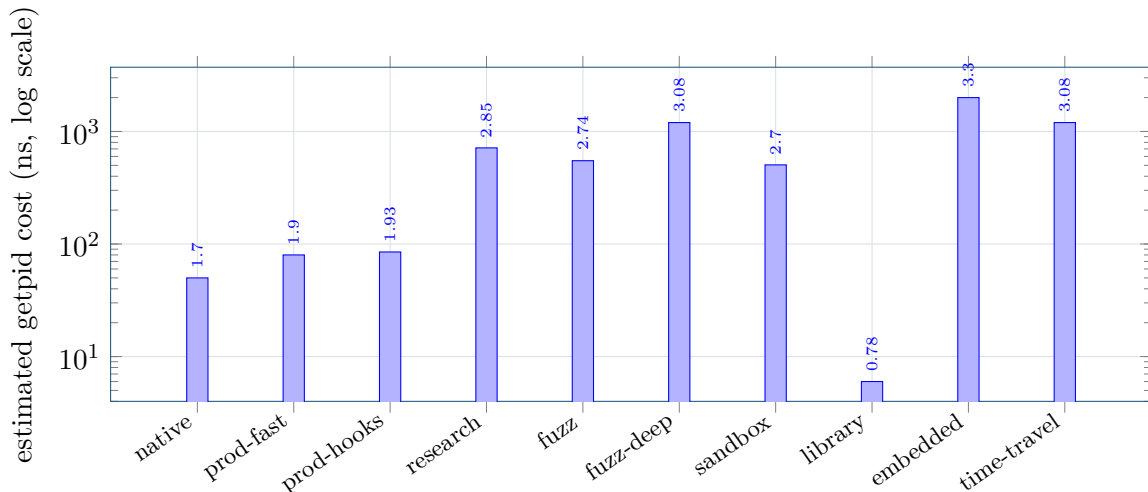


Figure 7: Profile cost model from the redesign docs, anchored to a 50 ns normal Linux `getpid` baseline. The value of profiles is the ability to span direct-call library mode through heavily instrumented research and time-travel builds without forking the tree.

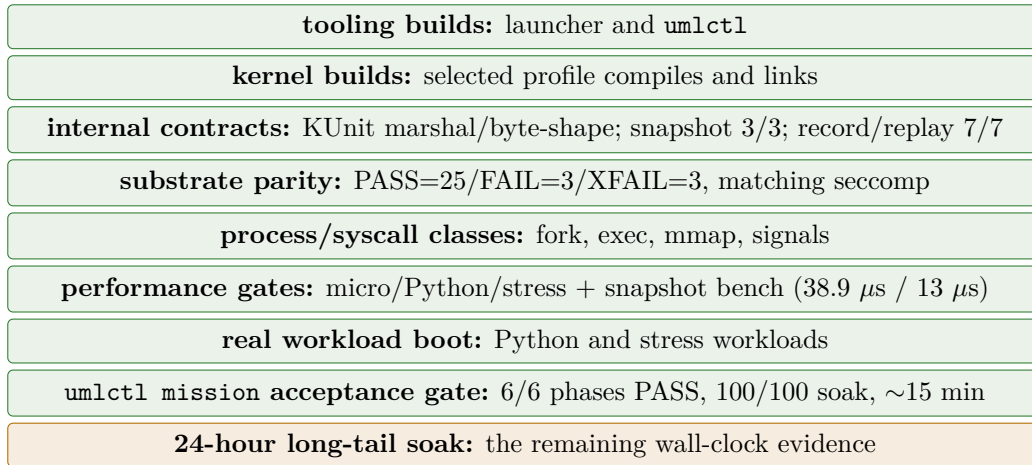


Figure 8: Validation ladder. Short gates protect the inner loop; the `umlctl mission` acceptance gate is the single-binary verdict; sustained 24-hour validation is the long-tail evidence reserved for milestone candidates.

6.2 Validation Ladder

6.3 umlctl mission Acceptance Gate (2026-05-19)

The first-class `umlctl mission` command replaces the ad-hoc soak invocation. One command, six fail-fast phases, a binary verdict, and a JSON scoreboard. Total wall-clock is ~15 min including an 8-min diverse-workload soak.

The mission gate is designed so a *single* regression in any phase — a flipped KUnit case, a missed performance target, a substrate gate break, an env-var-knob malfunction, a panic during soak, or a misconfigured host that the preflight catches — produces `MISSION_FAILED phase=<N>` on stderr and a non-zero exit. The Phase 5 gate explicitly fails on any of panics, “Kernel mode signal 7” (/dev/shm exhaustion), or “high-cr2” (the residual user-fault flake class) regardless of the aggregate pass rate.

6.4 Earlier Sustained Validation Pilot (2026-05-14)

The first post-T57 short pilot remains in the record as the chronological predecessor of the mission gate:

This pilot validated daemon mode, threshold-trip behaviour, Wilson summary generation, and sustained `kvm-v2` workload rotation. The three failures were `iocheck/seccomp` timeout shape, not `kvm-v2` regressions. The `umlctl mission` gate above subsumes this pilot as the canonical acceptance artefact.

7 Performance and Benchmarks

7.1 Benchmark Method

The post-gadget cross-host benchmark used one kernel build, one benchmark bundle, the same orchestrator, performance governor, and the same three tiers across eight lab hosts:

- **micro:** `getpid-loop`, `RDTSC-bracketed`, pure trap-mechanism cost.
- **py:** canned Python startup/mixed-syscall workload.
- **stress:** `mt-mini SMP T=8, ncpus=4`, strict memory verification.

7.2 Native Baseline Context

The cross-host charts below use same-host speedup over `seccomp` because that ratio cancels most CPU-frequency and microarchitecture noise. For intuition, the profile cost model also carries a normal Linux `getpid` reference of 50 ns, shown here as 1.0x.

7.3 Micro Tier

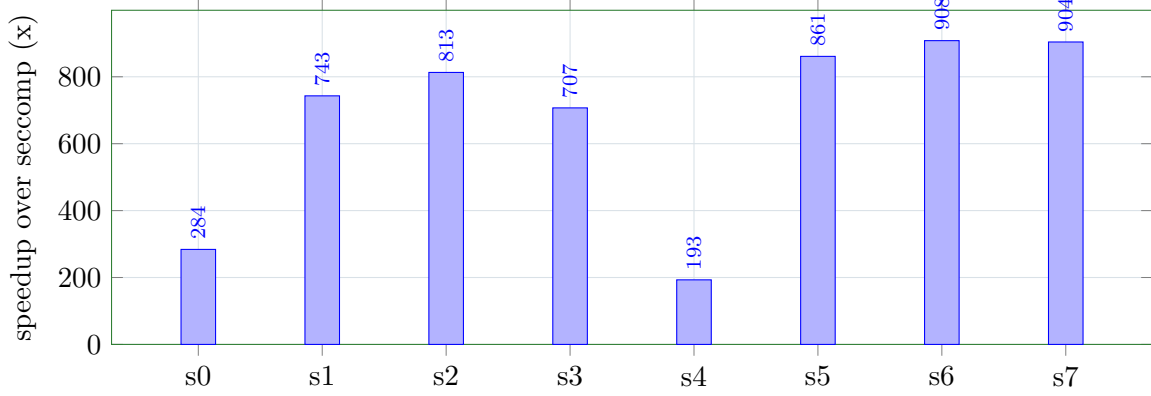


Figure 9: Post-gadget micro speedup. Trivial syscall dispatch is 193–908x faster than `seccomp` across the eight hosts.

The micro result is the clearest evidence that the LSTAR gadget changed the backend’s cost class. Before the gadget, a trivial syscall paid a host-exit round trip. After gadget revival, the measured `kvm-v2` cost is roughly 87–153 cycles on the tested machines, while `seccomp` remains tens of thousands of cycles per call.

7.4 Application and Stress Tiers

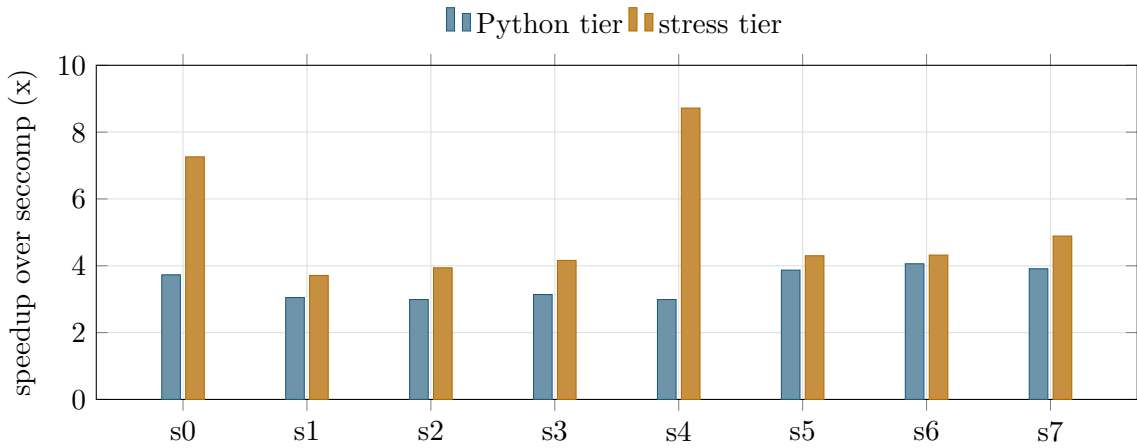


Figure 10: Application and stress tiers. `kvm-v2` wins on every host in both tiers.

The Python tier matters because it turns the micro win into an application-level win: 2.99–4.06x faster than `seccomp`. The stress tier matters because it combines performance with correctness: the run reported zero strict memory failures and zero verify failures on every host.

7.5 Detailed Cross-Host Table

7.6 Snapshot Benchmark

The Phase 5 snapshot bench harness (`kvm_v2_snapshot_bench=N` on the kernel cmdline, or via `/sys/kernel/debug/um/kvm_v2_snapshot_bench`) runs an N -iteration capture-once + restore-full- N -times cycle on the in-tree `kvm-v2` snapshot primitives.

The 13 μ s median restore is striking. The memo-12 target of “sub-ms iteration” was set conservatively before the v2 vCPU pool and identity-mapped memslot landed; v2’s design intent now shows up in the bench: no per-restore TLB shutdown, the memslot memcpy is a single host-VA-to-host-VA copy, and the regs/sregs push is a fixed-size ioctl chain.

7.7 Performance Interpretation

The performance story has four layers:

- **Mechanism win.** The micro tier proves that in-guest fast-path dispatch removes most seccomp signal/stub cost for trivial syscalls.
- **Workload win.** The Python tier proves that real userland startup benefits from those trivial syscalls becoming cheap.
- **Concurrency win.** The stress tier is mmap/page-fault heavy, so the gadget is less directly relevant; the continuing 3.71–8.72x speedup comes from the broader vCPU and SMP design.
- **Snapshot win.** The snapshot capture/restore numbers are within the sub-ms iteration budget the memo-12 design called for, by orders of magnitude. Record/replay sessions can now realistically checkpoint at every observed-syscall boundary without budget pressure.

The correct benchmark unit is the ratio against seccomp on the same host, not the raw absolute latency across machines. That choice removes most CPU-frequency and microarchitecture noise and keeps the claim tied to a backend decision. For snapshot the comparison is against the memo-12 design budget, which is the closest thing to a “one host, one budget” reference for snapshot/replay work.

8 Remaining Work and Upstream Path

8.1 Immediate Work

8.2 Snapshot, Replay, and Time-Travel Status

The “time machine” story has substantially advanced since the previous reporting cutoff. Snapshot and record/replay ports are landed in tree; the remaining work is host-side wiring rather than design.

8.3 Residual Risks

Two narrow residual flake classes are tracked as P2: an mt-mini page-recycling class and a high-yield mt-yieldonly RIP-corruption class. They are not current sustained-validation blockers and do not block snapshot/record-replay resumption, but they should remain visible in the report because they represent exactly the kind of concurrency state drift the state-audit method was built to catch.

8.4 Upstream Queue

The upstream strategy is deliberately staged. Small, independent subsystem patches go first. The backend abstraction is the tentpole. Static-key hot paths and profile work follow. The KVM backend is last because it is the largest technical and review burden.

8.5 Main Message for Maintainers

The upstream path makes three narrower claims:

1. UML already has multiple backend mechanisms; making the backend contract explicit is a cleanup and an enabling move.
2. static-key hooks are the right kernel-native way to make instrumentation runtime-flippable without charging every profile.
3. The `kvm-v2` evidence is now strong enough to submit. The Round-14 cache-flake closure (SMP-T73) was the last substantive correctness gap; the snapshot and record/replay v2 ports landed in tree with full KUnit coverage; host resource controls (memo 52, SMP-T78–T84) shipped as declarative TOML + cgroup v2 in `umlctl`; and the single-command `umlctl mission` acceptance gate gives a binary, reproducible 15-minute verdict for any reviewer to run locally.

A Source Map

Table 23: Primary source material used for this report.

Path	Role in the report
<code>00-vision.md</code>	Vision, non-goals, success criteria, stakeholder framing.
<code>README.md</code>	Top-level project layout and high-level architecture summary.
<code>STATUS.md</code>	Current state, phase ledger, workstream status, open items.
<code>01-architecture/three-layers.md</code>	Backend/gate/wrap architecture.
<code>01-architecture/data-flow.md</code>	Per-profile syscall cost model and profile comparison.
<code>02-workstreams/README.md</code>	Workstream decomposition and critical path.
<code>02-workstreams/D-kvm-backend/README.md</code>	KVM backend motivation and design framing.
<code>03-architecture-review-2026-04-27/00-synthesis.md</code>	v1 root-cause analysis: KVM state ownership, signal masking, and shadow page-table lessons.
<code>02-workstreams/D-kvm-backend/25-v2-restart-guide.md</code>	Mechanical archive/restart plan that records why v1 was retired and what v2 had to change.
<code>02-workstreams/D-kvm-backend/26-v2-implementation-plan.md</code>	Detailed v2 implementation plan and milestone diary.
<code>02-workstreams/D-kvm-backend/state-audit/</code>	State ownership audit, bug chronology, and recent fixes.
<code>02-workstreams/D-kvm-backend/state-audit/26-feature-enablement-checklist.md</code>	SMP-T77 forward-looking checklist: what changes when a new CPUID bit is un-masked (AVX-512 / AMX / PKRU / CET).
<code>02-workstreams/D-kvm-backend/26-snapshot-v2-port.md</code>	Snapshot port design memo (Phase 1–6 sub-sequencing).
<code>02-workstreams/D-kvm-backend/27-record-replay-v2-port.md</code>	Record/replay port design memo (Phase 1–7 sub-sequencing).
<code>02-workstreams/D-kvm-backend/bench-cross-host-2026-05-04-postgadget.md</code>	Cross-host benchmark data and methodology.

Path	Role in the report
02-workstreams/D-kvm-backend/phase-J-soak-2h-2026-05-14.md	First sustained-validation pilot result (subsumed by <code>umlctl mission</code> as the canonical acceptance artefact).
08-future-phases/50-kvm-v2-django-flake-investigation-summary.md	Round 1–14 cache-flake investigation summary; root-cause narrative for SMP-T73.
08-future-phases/51-kvm-v2-django-flake-deeper-debug-plan.md	Round-13 deeper-debug plan that fed the Round-14 close.
08-future-phases/52-uml-host-resource-controls.md	Host resource controls plan (SMP-T78–T84 implementation + T85–T87 deferred design).
03-profiles/README.md	Profile matrix and estimated per-profile cost model.
05-validation/	Validation tiers, benchmarks, conformance, CI, and quality strategy.
upstream-patches/SUBMISSION-QUEUE.md	Sequenced LKML/upstream queue.
tools/testing/selftests/um/	Concrete gate, parity, soak, benchmark, and reproducer harnesses.
tools/testing/selftests/um/soak/*.toml.template	11 soak workload templates, each with a default <code>[host_resources]</code> block (memo 52).
tools/testing/selftests/um/snapshot-kvm-smoke/	v2 snapshot KUnit smoke harness (3/3 cases).
tools/testing/selftests/um/kvm-snapshot-bench/	Capture/restore latency benchmark gate.
tools/testing/selftests/um/kvm-record-smoke/	v2 record/replay KUnit smoke harness (7/7 cases).
tools/uml/uml-launcher/	Host-side launcher and <code>umlctl</code> machinery.
tools/uml/uml-launcher/src/bin/umlctl/mission.rs	SMP-T96 mission-accomplished subcommand: 6-phase acceptance gate, JSON scoreboard, binary verdict.
tools/uml/uml-launcher/src/bin/umlctl/cgroup.rs	SMP-T83 cgroup v2 per-instance subgroup management.
tools/uml/uml-launcher/src/bin/umlctl/preflight.rs	SMP-T84 preflight resource verification module.
arch/um/backend/kvm-v2/snapshot.c	Snapshot port implementation (Phase 1–6) including cmdline + debugfs bench harness.
arch/um/backend/kvm-v2/record.c	Record/replay port implementation (Phase 1–7).
arch/um/backend/kvm-v2/test_snapshot.c	Snapshot KUnit suite.
arch/um/backend/kvm-v2/test_record.c	Record/replay KUnit suite.
arch/um/os-Linux/process.c	Host-side <code>os_map_memory</code> carries the memo-52 SMP-T71/T78/T79/T81 levers.
arch/um/os-Linux/main.c	Host-side <code>os_main</code> carries the memo-52 SMP-T71/T80/T82 levers.

B Reporting Data Files

The report and deck share normalized data under `Documentation/virt/uml/redesign/report-presentation/data/`. At this cutoff the important files are:

- `bench_speedups.csv` and `bench_ratios.csv` for the eight-host post-gadget benchmark.
- `soak_summary.csv` for the 2026-05-14 two-hour sustained-validation pilot (now subsumed by the `umlctl` mission acceptance gate as the canonical artefact, but retained for chronology).
- `profile_getpid.csv` for profile cost visualization.
- `workstreams.csv`, `validation_ladder.csv`, `upstream_queue.csv`, and `status_kpis.csv` for status tables.

Reproducing the canonical acceptance artefact:

```
$ umlctl mission --kernel <path-to-uml-binary>
[Phase 1] PASS kunit          ( ~5s)  -- 10/10 KUnit cases PASS
[Phase 2] PASS bench          ( ~3s)  -- capture=35us median=13us
[Phase 3] PASS substrate      ( ~4s)  -- cpython-tier0 PASS
[Phase 4] PASS host_resources ( ~1s)  -- env knobs applied silently
[Phase 5] PASS soak           (~14m)  -- 100/100 PASS; 0 panics
[Phase 6] PASS diagnostic      (~<1s)  -- metadata captured
MISSION_ACCOMPLISHED in 867s (kernel=9ccbf5e7bb58, host=<H>)
```

JSON scoreboard at `<out>/scoreboard.json`. `--quick` skips Phase 5 (the diverse-workload soak) for ~12-second dev iteration. `--continue-on-fail` runs all six phases regardless to collect every signal in one pass.

References

- [1] Jeff Dike. “A User-mode Port of the Linux Kernel”. In: *Proceedings of the 4th Annual Linux Showcase and Conference*. USENIX / ALS 2000. 2000.
- [2] *User Mode Linux HOWTO*. https://docs.kernel.org/virt/uml/user_mode_linux_howto_v2.html. Linux kernel documentation.
- [3] *User Mode Linux*. <https://user-mode-linux.sourceforge.net/>. Project homepage.
- [4] *gVisor Architecture Guide: Platforms*. https://gvisor.dev/docs/architecture_guide/platforms/. gVisor documentation.
- [5] *Firecracker MicroVM*. <https://firecracker-microvm.github.io/>. Project documentation.
- [6] *Linux Kernel Library*. <https://github.com/lkl/linux>. Project repository.
- [7] *crosvm Book: Architecture Overview*. <https://crosvm.dev/book/architecture/overview.html>. Project documentation.
- [8] *syzkaller*. <https://github.com/google/syzkaller>. Project repository.
- [9] *Linux Kernel Documentation: Coding Assistants*. <https://docs.kernel.org/process/coding-assistants.html>. Linux kernel documentation.
- [10] *KVM API Documentation*. <https://docs.kernel.org/virt/kvm/api.html>. Linux kernel documentation.

Table 13: Selected recent `kvm-v2` correctness and performance closures.

Item	Closure	Why it matters
SMP-T41	Recover user RAX in EINTR-mid-PF-stub	Closed the true mt-mini byte-zero residual.
SMP-T54	SOCK_CLOEXEC and umlctl sysrq halt	Removed worker fd leak and init shutdown hang class.
SMP-T55	Per-vCPU FPU-dirty epoch	Restored perf-py-startup gate while preserving cross-task FPU guarantees.
SMP-T57 AVX/XSAVE	CR4.OSXSAVE, XCR0=0x7, CPUID AVX/XSAVE unmask	Turned stress-ng AVX #UD from a fragile mask problem into supported guest architectural state.
Gadget revival	LSTAR in-guest fast path	Collapsed trivial syscall cost from host-exit scale to roughly 90 cycles on measured hosts.
SMP-T68	LSTAR-gadget inverted bounds check (operand order)	Restored correctness of the gadget’s user-pointer bounds gate after a regression introduced by an earlier assembler-syntax change.
SMP-T71	MAP_POPULATE on physmem + opt-in MAP_LOCKED + unconditional mlockall(MCL_CURRENT MCL_FUTURE MCL_ONFAULT)	Eliminates first-touch page-fault latency and (with <code>setcap</code>) blocks host migration / reclaim of the UML physmem mapping. Foundation for memo 52’s resource-control plan.
SMP-T73	KVM_GET_FPU → KVM_GET_XSAVE at 5 per-task save/restore sites	<i>The R14 closure.</i> See §5.8. Five-week, fourteen-round investigation; 1–2% bytecode-page corruption under SMP traced to legacy 512-byte FXSAVE area missing YMM upper 128 bits after SMP-T57’s AVX un-mask.
SMP-T74 / T75 / T76	DEBUGREGS pinned-zero at vCPU create; per-task KVM_GET/SET_VCPU_EVENTS save/restore (iotrap_events); CPUID leaf 0xD sub-leaf 5/6/7/17/18 consistency mask	Latent-bug follow-ups identified during the R14 hardware-acceleration audit. Same architectural shape as T73 (per-host-CPU vCPU pool, cross-task state hand-off).
SMP-T77	Feature-enablement checklist memo (state-audit Layer 26)	Forward-looking: documents what changes when a new CPUID bit is un-masked (AVX-512 / AMX / PKRU / CET / AVX-VNNI / AVX10+APX). Prevents re-walking the T26 → T57 → T73 audit gap on the next feature exposure.
SMP-T78–T82	Host-side env-var-gated knobs: MADV_UNMERGEABLE on physmem (always on); UM_THP=off/on; UM_OOM_SCORE_ADJ=N; UM_HUGEPAGES=2M/1G; UM_KVM_V2_CPU_AFFINITY=list	Memo 52 §1–§2. Predictable physmem behaviour, host-memory budget control, hugepage-backed TLB win, CPU-set confinement.
SMP-T83 / T84	umlctl [host_resources] TOML section + cgroup v2 per-instance subgroup + preflight resource verification	Declarative form of T78–T82 plus cgroup <code>memory.max</code> / <code>cpu.max</code> / <code>pids.max</code> ; preflight emits clear warnings (empty hugepage pool, un-walkable cgroup, missing <code>CAP_IPC_LOCK</code>).

Table 14: Shipped profile intent.

Profile	Backend default	Hooks	Sanitizers	Purpose
prod-fast	KVM > seccomp	none	none	Minimum over-head production target.
prod-with-hooks	KVM > seccomp	compiled, off	none	Production artifact with post-failure observability.
research	seccomp	trace, kprobes	KASAN, UBSAN	Debug and exploit-reproduction profile.
fuzz	seccomp	KCOV, snapshot	KASAN	syzkaller and harness fuzzing.
fuzz-deep	seccomp	KCOV, snapshot, replay	KASAN, KCSAN	deeper race/replay investigations.
sandbox	seccomp-only	none compiled	none	Minimized TCB and debug surface.
library	direct call	n/a	optional KASAN	LKL-style harness integration.
embedded	ptrace	none	none	Old-host fallback.
time-travel	seccomp	trace, time-travel	KASAN	deterministic clock and replay.

Table 15: Mission-accomplished run, kernel 9ccbf5e7bb58, host server7, 2026-05-19.

#	Phase	Verdict	Duration	Summary
1	KUnit	DONE	5.0 s	10/10 cases PASS (snapshot 3 + record 7)
2	Bench	DONE	2.5 s	capture 35 μ s, median restore 13 μ s (targets <50 ms / <1 ms)
3	Substrate	DONE	3.5 s	cpython-tier0 PASS
4	Host resources	DONE	1.4 s	UM_THP / UM_OOM_SCORE_ADJ / UM_KVM_V2_CPU_AFFINITY applied silently; backend banner clean
5	Diverse soak	DONE	855.0 s	100/100 PASS across memcheck + ioccheck + stress-ng + tier1-pylibs + django-loopback-none; 0 panics / 0 SIGBUS / 0 high-cr2
6	Diagnostic	DONE	0.4 s	kernel SHA-256, host, hugepage pool sizing, cgroup v2 state captured
MISSION_ACCOMPLISHED		867.7 s total wall-clock (14.5 min)		

Table 16: First post-T57 sustained-validation pilot, 2026-05-14, retained for chronology.

Workload	Backend	n	pass	fail	pass rate
memcheck	kvm-v2	60	60	0	100.00%
memcheck	seccomp	60	60	0	100.00%
iocheck	kvm-v2	60	60	0	100.00%
iocheck	seccomp	60	57	3	95.00%
stress-ng	kvm-v2	40	40	0	100.00%
stress-ng	seccomp	40	40	0	100.00%
tier1-pylibs	kvm-v2	40	40	0	100.00%
tier1-pylibs	seccomp	40	40	0	100.00%
kvm-v2 aggregate		200	200	0	100.00%
Overall aggregate		400	397	3	99.25%

Table 17: `getpid` context relative to a normal host syscall baseline.

Path	Model cost	Relative to normal	Reading
Normal Linux host syscall	50 ns	1.0x	Human-scale baseline for a trivial syscall on the host.
kvm-v2 prod-fast	80 ns	1.6x	Same cost class as the host baseline for gadget-covered calls.
kvm-v2 cross-host gadget	87–153 cycles	mechanism tier	Raw cycles vary by host; the robust claim is same-host speedup over seccomp .
seccomp profiles	505–1,200 ns	10–24x	Expected for research, fuzz, sandbox, and time-travel profiles where instrumentation or isolation dominates.
ptrace fallback	2,000 ns	40x	Old-host fallback, useful for compatibility rather than speed.
direct library call	6 ns	0.12x	LKL-style mode removes the syscall boundary; it is not a faithful ring-split kernel target.

Table 18: Post-gadget speedup over seccomp.

Host	CPU	micro	Python	stress
s0	i9-12900K	284x	3.73x	7.26x
s1	Xeon E3-1225 v6	743x	3.05x	3.71x
s2	Xeon E3-1225 v5	813x	2.99x	3.94x
s3	Xeon W-2123	707x	3.14x	4.16x
s4	i5-12600K	193x	2.99x	8.72x
s5	Ryzen 7 7840HS	861x	3.87x	4.30x
s6	Ryzen 7 7840HS	908x	4.06x	4.32x
s7	Ryzen 7 7840HS	904x	3.91x	4.89x

Table 19: Snapshot bench result, kernel 9ccbf5e7bb58, AMD Ryzen 7 7840HS, N=64.

Metric	Value	Memo-12 target
Capture (one-shot)	35–39 μ s	< 50 ms ($\sim 1300\times$ under)
Restore (median)	13 μ s	< 1 ms ($\sim 75\times$ under)
Restore (p95)	16–19 μ s	—
Restore (min)	12.5 μ s	—
Restore (max)	19–22 μ s	—
Mode	full	“full” = memslot copy ran; falls back to “regs-only” on kvmalloc -ENOMEM

Table 20: Next work at the reporting cutoff.

Item	State	Notes
24h long-tail soak	IN PROGRESS	The <code>umlctl mission</code> 15-min acceptance gate passes 6/6 phases including 100/100 soak. The wall-clock 24-hour run remains as the long-tail evidence; the rig is in place.
Series 7 (kvm-backend) LKML submission	IN PROGRESS	Cover letter is drafted (414 $\rightarrow$ 900 lines pre-R14). Now backend-layer unblocked by R14 closure + the <code>umlctl mission</code> acceptance gate; needs cover-letter refresh to record the snapshot+RR landings and submission.
Record/replay signal wiring	host-side PLANNED	The log shape, observe/consume API, KUnit suite, and replay-side gating all proven via memo 27 Phase 4–7. Remaining work is the host-side signal-interception that turns a real SIGALRM into an <code>observe_sigalrm</code> call.
Tier 3 host resources (T85/T86/T87)	PLANNED	SCHED_FIFO RT, NUMA mbind, 0_DIRECT for hostfs. Memo 52 §3 keeps the design; implementation opens only when a concrete operator question demands it.
Snapshot ELF64-core export (#181)	PLANNED	The snapshot port is done; the gdb/crash-tool export is a separate follow-on.
syzkaller vm/uml binding	planned external	The in-tree forkserver surface is pinned (snapshot v1 C-09); the remaining <code>pkg/vm/uml</code> Go driver belongs in the syzkaller repository.
LTP curation finalization	PLANNED	The <code>ltp-runner</code> soak template exists; the curated KEEP/SKIP list and UML-appropriate runner refinements still need committing.
AVX-512 / AMX / PKRU / CET un-mask	PLANNED	Memo state-audit Layer 26 (SMP-T77) preserves the checklist; each future un-mask follows it. AVX-VNNI and AVX10 require no XSAVE-pipe changes (T73’s XSAVE plumbing covers them).

Table 21: Snapshot, replay, and time-travel state.

Surface	State	Reading
Snapshot/forkserver v1	DONE	C-09 (legacy): <code>um_snapshot_ready()</code> point, AFL-compatible fds 198/199, <code>/sys/kernel/um/state_version</code> , launcher fd plumbing, and <code>snapshot-smoke</code> coverage.
v1 ceiling	known limit	Status is reported as zero, payloads must be short and non-blocking, there is no on-disk snapshot, and v1 is not SMP-capable. These are documented limits, not hidden claims.
KVM-aware snapshot v2 (#168)	DONE	Memo 26-snapshot Phase 1–6 landed: <code>kvm_v2_snapshot_alloc/capture/restore</code> surface (regs-only and full), KUnit 3/3 cases including the Phase-4 cross-task semantics test, <code>kvm_v2_snapshot_bench</code> cmdline + debugfs harness (Phase 5), and the <code>snapshot-kvm-smoke</code> selftest re-plumbed against v2 case names (Phase 6). Bench numbers: 38.9 μ s capture, 13 μ s median restore.
KVM-aware record/replay v2 (#169)	DONE	Memo 27 Phase 1–7 landed: <code>kvm_v2_record_*</code> state machine, static-key hot-path gate, observe/consume primitives for SYSCALL + RDTSC + SIGALRM kinds, anonymous-union replay-entry payload, LSTAR-gadget <code>KVM_V2_GADGET_OFF_RECORD</code> bypass byte, KUnit 7/7 cases (basic / state / observe / strict_replay / gadget_bypass / rdtsc / sigalrm), and the <code>kvm-record-smoke</code> selftest re-plumbed against v2 case names.
Time-travel profile	profile exists	<code>uml/time-travel</code> selects seccomp-only, UP, <code>CONFIG_UML_TIME_TRAVEL_SUPPORT</code> , and debug surfaces. The runtime hook gate exists; today its slow path is still a counter stub.
Record/replay host wiring	PLANNED	The log shape and replay-side gating are KUnit-proven. The remaining work is the host-side signal-interception path that turns a real SIGALRM into an <code>observe_sigalrm</code> call, plus the trap-on-RDTSC VMCS programming (memo 27 §6 Q1 spike).
Snapshot ELF64-core export (#181)	PLANNED	Lift to a gdb/crash-tool-loadable artefact, gated on snapshot v2 in tree (now complete).
Syzkaller binding	planned external	The in-tree forkserver surface is pinned; the remaining <code>pkg/vm/uml</code> Go driver belongs in the syzkaller repository.

Table 22: Sequenced upstream queue.

#	Series	Size	Subsystem	Status
1	<code>bpf-hygiene-v1</code>	2 patches	BPF	ready
2	<code>kmsan-arch-callback-rfc</code>	1 RFC patch	mm/kmsan	ready
3	<code>ftrace-notrace-generic-v1</code>	1 patch	tracing/ftrace	ready
4	backend ops abstraction RFC	about 12 patches	arch/um	drafted
5	static-key hot paths	about 6 patches	arch/um	drafted
6	per-profile C-series	about 20 patches	arch/um plus sub-systems	blocked by Series 7 ordering
7	KVM backend series	about 15 patches	arch/um plus KVM	cover-letter drafted (414 → 900 lines); backend-layer unblocked by R14 closure + <code>umlctl</code> mission acceptance gate; needs cover-letter refresh + submission